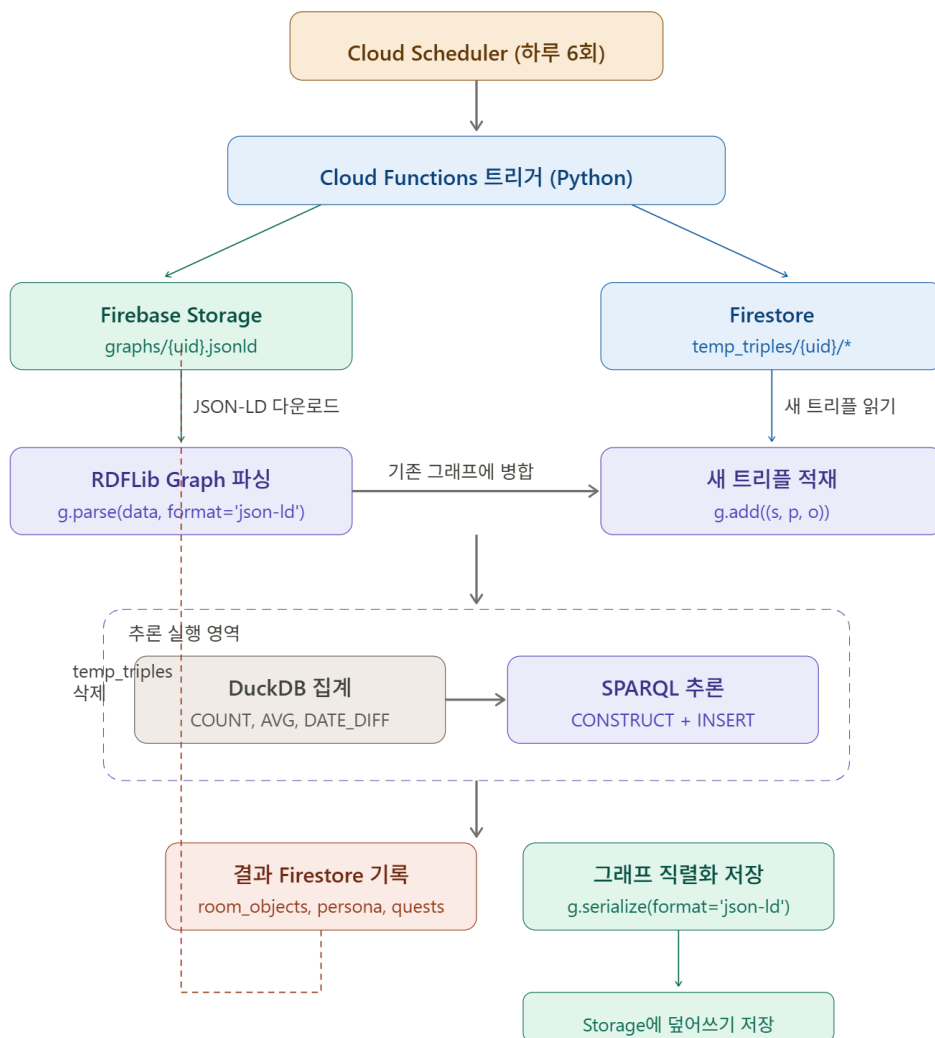


## 4. 온톨로지 그래프 로드/ 5. Duck DB + SPARQL 추론

🌟 고정하기 ☐

### 4. 온톨로지 그래프 로드 과정

Cloud Functions는 Stateless이기 때문에 매 batch마다 Storage에서 그래프를 로드하고, 추론 후 다시 저장하는 사이클을 반복합니다.



이 다이어그램이 그래프 로드/저장 사이클의 전체 그림입니다. Cloud Functions는 Stateless이므로 매 batch마다 Firebase Storage에서 `graphs/{uid}.jsonld` 파일을 다운로드하여 `g.parse()` → Firestore의 `temp_triples` 에서 새 트리플을 읽어 `g.add()` → 추론 실행 → 결과 기록 → `g.serialize()` 로 다시 Storage에 덮어쓰기하는 사이클을 반복합니다.

## 5. DuckDB와 SPARQL의 역할 분담

실제 데이터가 어떻게 흘러가며 각 엔진이 무엇을 처리하는지가 포인트입니다.

#### Phase 1 — DuckDB 집계 분석

RDFlib 그래프의 트리플을 DuckDB 인메모리 데이터베이스로 내보낸 뒤, SQL로 빈도/시간/교차 패턴을 집계한다. SPARQL은 집계 함수 (COUNT, AVG, DATE\_DIFF 등)가 약하므로, 수치 분석은 DuckDB가 전담한다.

예시 1: 카페 방문 빈도 집계

```
SELECT object AS place,
       COUNT(*) AS visit_count,
       COUNT(DISTINCT WEEK(date)) AS active_weeks
FROM   triples
WHERE  predicate = 'lp:visited'
AND    object LIKE '%cafe%'
AND    date >= CURRENT_DATE - INTERVAL '30 DAY'
GROUP BY object
HAVING COUNT(*) >= 5;
```

DuckDB 출력    cafe\_gangnam: 12회/월, 4주 연속 — 빈도 임계값 초과

예시 2: 수면-운동 시간 간격 분석

```
SELECT a.date,
       a.value AS sleep_hours,
       b.value AS step_count,
       DATEDIFF('day', a.date,
               (SELECT MIN(c.date) FROM triples c
                WHERE c.predicate = 'lp:stepCount'
                 AND c.value > 5000
                 AND c.date > a.date))
       ) AS days_until_exercise
FROM   triples a
JOIN   triples b ON a.subject = b.subject
       AND a.date = b.date
WHERE  a.predicate = 'lp:sleepHours'
AND    b.predicate = 'lp:stepCount'
AND    CAST(a.value AS FLOAT) < 6.0
ORDER BY a.date DESC LIMIT 14;
```

DuckDB 출력    수면 6h 미만인 날 → 평균 2.3일 뒤까지 운동 5000보 미  
달

예시 3: 야간 음악 장르 교차 분석

```
SELECT genre,
       ROUND(COUNT(*) * 100.0 / SUM(COUNT(*) OVER(), 1) AS pct
FROM   triples
WHERE  predicate = 'lp:genre'
AND    EXTRACT(HOUR FROM timestamp) >= 22
GROUP BY genre
ORDER BY pct DESC;
```

DuckDB 출력    22시 이후: ballad 66%, jazz 18%, pop 14%

DuckDB 집계 결과를 Python dict로 SPARQL에 전달 ↓

#### Phase 2 — SPARQL 의미 추론

DuckDB의 수치 분석 결과를 바탕으로, RDFlib에서 SPARQL CONSTRUCT 쿼리를 실행하여 새로운 의미 트리플(관심사, 인과 체인, 건강 물레그 등을) 그래프에 삽입한다. 규칙 기반이므로 동일 입력에 동일 출력이 보장된다.

규칙 1: 관심사 노트 생성 (CONSTRUCT)

```
CONSTRUCT {
  ?user lp:hasInterest ?interest .
  ?interest rdf:type lp:InterestNode .
  ?interest lp:label ?label .
  ?interest lp:confidence ?conf .
  ?interest lp:derivedFrom ?evidence .
}
WHERE {
  # DuckDB 결과기 Python에서 변환된 값
  BIND(user:001 AS ?user)
  BIND(interest:coffee_lover AS ?interest)
  BIND('커피 애호가' AS ?label)
  BIND(0.92 AS ?conf)
  BIND('cafe visit 12/month' AS ?evidence)
}
```

SPARQL 출력    (user:001, lp:hasInterest, interest:coffee\_lover) + 메타데이터 4개 트리플

규칙 2: 인과 추론 체인 생성

```
CONSTRUCT {
  ?chain rdf:type lp:CausalChain .
  ?user lp:hasCausalChain ?chain .
  ?step1 lp:causedBy ?step2 .
  ?step2 lp:causedBy ?step3 .
  ?chain lp:derivedFrom ?evidence .
  ?chain lp:suggestion ?action .
}
WHERE {
  # DuckDB: 수면<6h 후 평균 2.3일 운동 부족 패턴 확인
  BIND(chain:sleep_exercise_001 AS ?chain)
  BIND(user:001 AS ?user)
  BIND('수면 부족 (5.2h)' AS ?step1)
  BIND('다음 날 운동 스킵' AS ?step3)
  BIND('주간 활동량 30% 감소' AS ?step2)
  BIND('sleepDx → exercise skip correlation' AS ?evidence)
  BIND('오늘은 11시 전에 자보는 건 어때?' AS ?action)
}
```

SPARQL 출력    CausalChain 1건 + causedBy 관계 2건 + suggestion 1건

규칙 3: 빈 노트 감지 → 퀘스트 생성 (ASK + CONSTRUCT)

```
# 먼저 ASK로 빈 노트 감지
ASK {
  ?user lp:visited ?place .
  FILTER NOT EXISTS { ?visit lp:companion ?anyone }
}
# True이면 퀘스트 생성
CONSTRUCT {
  ?quest df:type lp:Quest .
  ?quest lp:type 'data_완전' .
  ?quest lp:question '오늘 카페 누구랑 갔어?' .
  ?quest lp:triggeredBy ?place .
}
```

퀘스트 생  
성    companion 빈 노트 → "오늘 카페 누구랑 갔어?" 퀘스트 자동 생  
성

추론 완료 → 결과 분배 ↓

#### Phase 3 — 결과 분배 및 그래프 저 장

추론으로 생성된 트리플들을 목적에 따라 분배하고, 경신된 그래프를 직렬화하여 Storage에 다시 저장한다.

Firestore    room\_objects — Persona → RoomObject 매핑 규칙에 따라 오브젝트 배치 정보 기록

Firestore    persona — users/uid에 최신 Persona JSON 백업 (덮어쓰기)

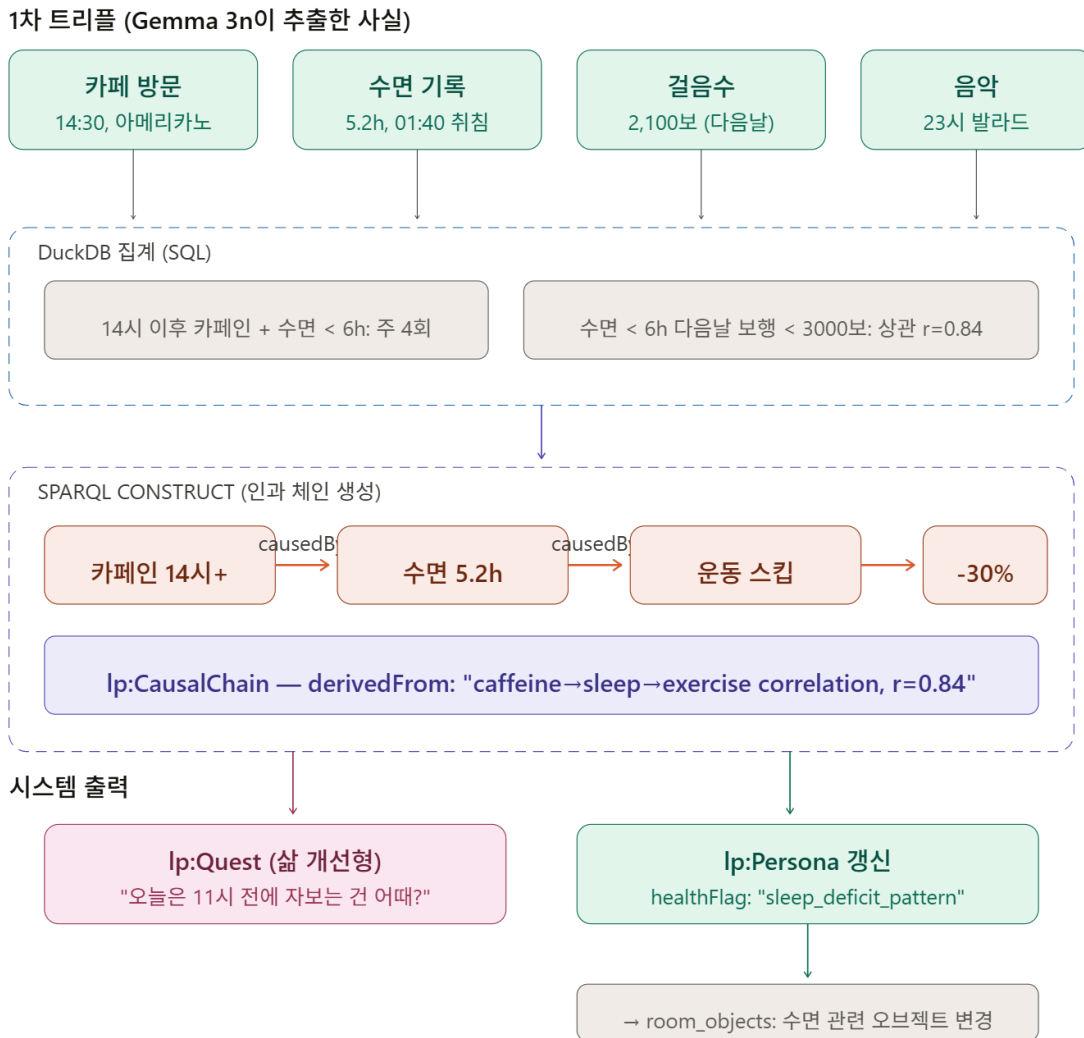
Firestore    quests — 빈 노트 감지 퀘스트 + 실행 개선 퀘스트 기  
록

Storage    graphs/uid.jsonld — 경신된 전체 그래프 직렬화 저장

삭제    temp/triples/uid/\* — 사용 완료된 임시 트리플 즉시 삭제

## 예시

DuckDB 집계 결과가 어떻게 SPARQL 추론 규칙으로 연결되어 최종 출력을 만들어내는 지



**다이어그램 1 — 그래프 로드/저장 사이클**은 Cloud Functions가 Stateless라는 제약을 어떻게 극복하는지를 보여줍니다. Cloud Scheduler가 하루 6회 트리거하면, Cloud Functions가 Firebase Storage에서 `graphs/{uid}.jsonld` 를 다운로드하여 RDFLib Graph 객체로 파싱하고, Firestore `temp_triples` 에서 새 트리플을 읽어 기존 그래프에 병합합니다. 추론 완료 후 결과를 Firestore에 기록하고, 갱신된 그래프를 다시 JSON-LD로 직렬화하여 Storage에 덮어쓰기 저장합니다. 사용이 끝난 `temp_triples` 는 즉시 삭제됩니다.

**다이어그램 2 — DuckDB + SPARQL 역할 분담**은 추론 실행의 핵심 메커니즘입니다. Phase 1에서 DuckDB가 SQL로 빈도 집계(`COUNT`), 시간 간격 분석(`DATE_DIFF`), 교차 패턴

( `JOIN + GROUP BY` )을 수행하고, Phase 2에서 이 수치 결과를 Python 변수로 바인딩하여 SPARQL `CONSTRUCT` 쿼리가 새로운 의미 트리플(관심사 노드, 인과 체인, 건강 플래그)을 생성합니다. 빈 노드 감지는 `ASK` 쿼리로 수행하여 퀘스트를 자동 생성합니다.

**다이어그램 3 — 인과 추론 체인 추적**은 "카페인 14시 이후 섭취 → 수면 5.2h → 다음 날 운동 스킵 → 주간 활동량 30% 감소"라는 구체적 인과 체인이 1차 트리플 → DuckDB 상관 분석( $r=0.84$ ) → SPARQL `causedBy` 체인 생성 → Quest(수면 개선) + Persona(healthFlag 갱신) + RoomObject(수면 오브젝트 변경)로 이어지는 전체 흐름을 추적합니다.